



Контролисан развој уз AI

AI ради унутар архитектонских граница и проверљивог развојног тока

AI се уводи тек када постоји тестирано језгро

Модел не сме да замени архитектуру нити одговорност студента.

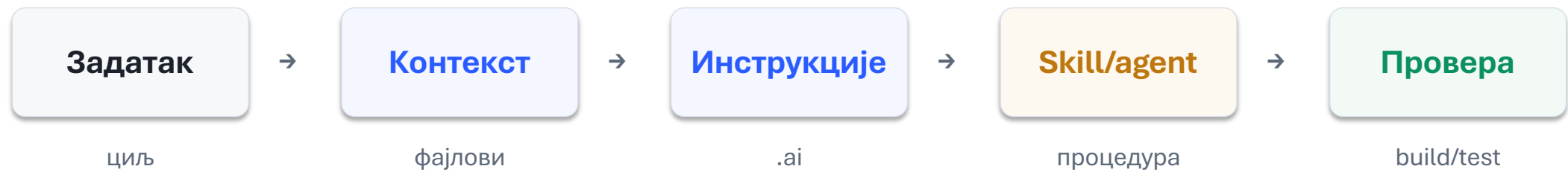
Циљ је да AI ради унутар граница solution-а и да сваки резултат остане проверљив стандардним развојним сигнаlima.

КЉУЧНА ПОРУКА

Build, tests и git diff остају извор истине.

Контролисан AI ток над solution-ом

Задатак мора имати циљ, контекст, ограничења и проверу.

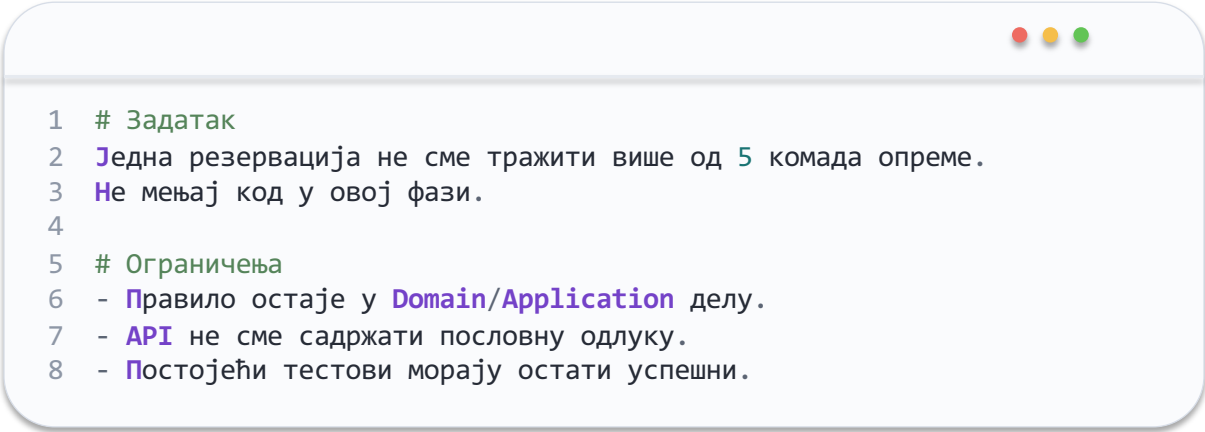


КЉУЧНА ПОРУКА

AI workflow је инжењерски процес, не разговор без граница.

Од нејасног захтева до проверљивог задатка

Уместо „среди резервације”, задатак каже шта се мења и како се доказује.



```
1 # Задатак
2 Једна резервација не сме тражити више од 5 комада опреме.
3 Не мењај код у овој фази.
4
5 # Ограничења
6 - Правило остаје у Domain/Application делу.
7 - API не сме садржати пословну одлуку.
8 - Постојећи тестови морају остати успешни.
```

Шта је важно

Прво анализа, затим измена.

Ограничења чувају архитектуру.

План се прегледа пре дозволе за писање.

Добар AI задатак има четири дела

Без њих се добија велики diff који је тешко бранити.

01

Понашање

шта се мења у систему

02

Границе

који слојеви смеју да се мењају

03

Доказ

које команде и тестови потврђују резултат

04

Излаз

план, ризици, фајлови и сценарији

Пројектне инструкције су верзионисана правила

У примеру постоји `.ai/AI_INSTRUCTIONS.md`.

Инструкције нису општи савет, већ конкретна правила архитектуре solution-a: Domain не зависи од других пројеката, Application дефинише портове, Infrastructure имплементира port-ове.

КЉУЧНА ПОРУКА

AI_USAGE чува сажет траг одлуке.

.ai/AI_INSTRUCTIONS.md као архитектонска граница

Правила треба да се могу проверити читањем references и покретањем тестова.

```
1  ## Архитектура
2  - Domain не зависи ни од једног другог пројекта.
3  - Application зависи само од Domain и дефинише port-ове.
4  - Infrastructure имплементира port-ове из Application
5    слоја.
6  - Api је composition root.
7  - MCP и guardrails не смеју постати зависност језгра.
```

Шта је важно

Инструкције су део репозиторијума.

Тим их прегледа као код.

Модел не сме тврдити да је проверио нешто што није извршио.

Skill раздваја процедуру од ad-hoc разговора

review-pull-request процедура има улазе, кораке и ограничење.

Review улога не мења код током review фазе, већ проверава захтев, diff, инструкције, тестове и архитектонске границе.

КЉУЧНА ПОРУКА

SRP важи и за агентски workflow.

Review процедура

Поступак се може поновити на више измена у solution-у.

- 01 Сажети очекивано понашање.
- 02 Проверити да ли diff излази из обима захтева.
- 03 Проверити Dependency Rule и границе модула.
- 04 Прегледати негативне и граничне сценарије.
- 05 Упоредити промењено понашање са тестовима.

Специјализоване улоге и најмање привилегије

Agent не треба да има више овлашћења него што задатак тражи.

Улога	Дозволе	Ограничење
architecture-reviewer	чита solution, source, инструкције и diff	не пише код
implementer	мења фајлове из усвојеног плана	не проширује захтев
test/review	покреће тестове и анализира резултат	не мења тест да сакрије грешку

Структурирана предаја задатка implementer-y

План ограничава простор измене.

```
1 {
2   "task": "Implement approved reservation quantity rule",
3   "constraints": [
4     "Keep business rule outside Api",
5     "Do not add Infrastructure dependency to
6       Domain/Application"
7   ],
8   "filesToConsider": [
9     "src/EquipmentReservation.Domain/InventoryItem.cs",
10    "tests/EquipmentReservation.Tests/ReservationTests.cs"
11  ]
12 }
```

Шта је важно

Фајлови су намерно ограничени.

Верификација је унапред дефинисана.

Ако се обим прошири, враћа се на анализу.

AI резултат није доказ

Модел може рећи да промена „изгледа исправно”, али то није завршетак рада.

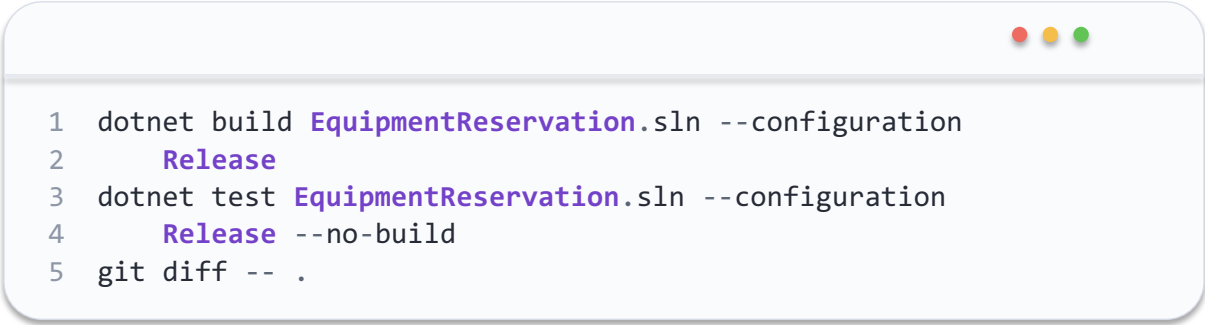
Завршетак задатка заснива се на стварним сигнаlima: успешан build, успешни NUnit тестови и преглед финалног diff-a.

КЉУЧНА ПОРУКА

Ако команда није извршена, не бележи се као успешна провера.

Минимална независна провера

Ове команде затварају AI измену.



```
1 dotnet build EquipmentReservation.sln --configuration
2   Release
3 dotnet test EquipmentReservation.sln --configuration
4   Release --no-build
5 git diff -- .
```

Шта је важно

Пад теста се не решава брисањем очекивања.

Diff не сме да додирује неочекиване слојеве.

Резултат команде је јачи од уверења модела.

AI_USAGE није транскрипт

Потребан је инжењерски траг, не цела конверзација.

01

Задатак

шта је тражено

02

Контекст

који фајлови и правила су коришћени

03

Одлука

шта је прихваћено или одбијено

04

Доказ

која провера је стварно извршена

Када зауставити AI ток

Понекад је најбоља одлука не наставити са моделом.

ЗАУСТАВИ

agent тражи .env или тајне

diff додирује неочекиване слојеве

тест пада без разумевања узрока

план проширује пословни захтев

НАСТАВИ

план је мали и проверљив

границе су испоштоване

тестови дају јасан сигнал

diff је разумљив за review

Одговорност тима остаје централна

AI може убрзати рад, али не може преузети одбрану.

01

Разумевање

студент објашњава решење

02

Провера

команде и тестови доказују
понашање

03

Одлука

тим прихвата или одбија
предлог

Рад на вежби

На копији EquipmentReservation solution-а изводи се један стварни AI развојни ток.

- 01 Задати малу промену пословног правила.
- 02 Прво користити architecture-reviewer само за анализу.
- 03 Implementer-у проследити усвојен план.
- 04 Покренути build/test и прегледати diff.
- 05 Унети сажет запис у AI_USAGE.md.

Кључна мера успеха

AI није вредан ако поремети проверљивост система.

3

стварна сигнала

Build, test и diff су минимум независне провере после AI измене.



Суштина Вежбе 6

Контролисан AI workflow није магија, већ дисциплинована употреба постојећих инжењерских сигнала.

Када су контекст, ограничења, улоге и провера јасни, AI може помоћи без рушења SOLID и Clean Architecture правила.

КЉУЧНА ПОРУКА

Следећи корак је формалнији приступ контексту кроз MCP.

За наредни термин

Припремити пројекат за MCP приступ контексту.

- 01 Идентификовати које информације AI сме само да чита.
- 02 Раздвојити resource од tool операција.
- 03 Одредити које команде смеју бити фиксно извршене.
- 04 Забележити шта никада не сме бити изложено као произвољан shell.
- 05 Понети build/test/diff сигнале као основу за MCP.